

Infrastructure as Code (IaC) Security Architecture

Tools	Terraform for provisioning and Ansible for configuration
Target Environment	Enterprise cloud infrastructure
Security Focus	Least privilege, secure state, secrets, policy and compliance
Name	Abdul Rehman
Weeks	6&7 Advance AI and Automation

1. Overview

Infrastructure as Code makes it easier to build and manage cloud infrastructure in a consistent way, but the code itself has to be secured. A mistake in Terraform or an unsafe configuration in Ansible can affect an entire environment. For this reason, the proposed architecture separates infrastructure provisioning from server configuration while putting security checks around both stages.

Terraform is responsible for creating the basic cloud infrastructure, such as the network, compute resources and storage. Ansible runs after Terraform and is used to apply operating-system settings, security hardening and compliance checks. Keeping these responsibilities separate makes the environment easier to manage and reduces the chance that one change will affect everything at once.

2. Terraform and Ansible Workflow

Terraform acts as the first stage of the deployment. Infrastructure changes are stored in a Git repository and should move through a CI/CD or GitOps pipeline instead of being applied directly from an engineer's workstation. This gives the team a clear record of what changed and who approved it.

The Terraform configuration is split into smaller modules instead of putting every resource into one large file. A root module can manage environment-specific settings and call reusable child modules for IAM and KMS, networking, storage and EC2. This also helps limit the blast radius when a change is made.

Once the infrastructure is created, Ansible handles the operating-system side. It can update packages, harden SSH, configure the firewall, enable fail2ban and auditd, and run compliance checks. The design uses AWS Systems Manager Session Manager for access rather than exposing servers through public IP addresses or maintaining a permanent bastion host.

3. Secure Terraform State

Terraform state needs special attention because it can contain sensitive information about the infrastructure. The state should therefore be stored remotely rather than kept on individual workstations.

In this architecture, the state is stored in an encrypted S3 bucket using AWS KMS. The bucket should only accept secure TLS connections and access should be limited to the CI/CD execution role. S3 versioning provides recovery points if the state is accidentally changed or damaged.

A DynamoDB table is used for state locking. This prevents two Terraform operations from modifying the same state at the same time and helps avoid race conditions. Together, encryption, restricted access, versioning and locking make the state backend safer and more reliable.

4. Secret Management and Access Control

Cloud credentials and other secrets should not be placed directly in source code, Git repositories or Terraform state. The preferred approach is to use short-lived credentials through OpenID Connect (OIDC). A CI/CD runner can authenticate with AWS and assume a limited IAM role instead of storing a permanent AWS access key.

Other secrets, such as database passwords, API tokens and certificates, can be retrieved at runtime from AWS Secrets Manager or HashiCorp Vault. For Ansible, sensitive variables can be protected with Ansible Vault rather than stored as plain text.

Access should follow the principle of least privilege. IAM roles should only have the permissions needed for the job they perform. This reduces the impact if an account, runner or application is compromised.

5. Security Checks and Policy Enforcement

Security checks should happen before infrastructure reaches staging or production. Local pre-commit checks can catch obvious problems such as leaked credentials, while tools such as tflint, Checkov, tfsec and Trivy can identify Terraform and cloud configuration issues during CI.

Policy as code adds another layer of protection. OPA/Rego or Terraform Sentinel policies can inspect Terraform plans and reject unsafe changes. For example, a plan should be blocked if it exposes management ports such as SSH to 0.0.0.0/0, creates unencrypted storage, or gives an IAM role unrestricted administrative permissions.

This shift-left approach is useful because problems are found before resources are created. A high or critical security finding can stop the pipeline instead of relying on someone to notice the issue after deployment.

6. Drift Detection, Rollback and Compliance

Infrastructure can change after it has been deployed, either because of an administrator using the cloud console or because of an unexpected process. The architecture therefore includes regular drift checks using Terraform plan. A scheduled plan can identify differences between the declared configuration and the live environment.

AWS Config and CloudTrail provide another source of visibility. Configuration changes and management events can be sent through EventBridge so that the security team can be alerted and, where appropriate, automated remediation can be triggered.

For rollback, the preferred approach is to use version-controlled Git releases. If a deployment causes a problem, the team can revert to a known-good version and redeploy it. If the Terraform state itself becomes corrupted, versioned S3 state snapshots can be used for recovery.

Compliance is checked continuously against standards such as the CIS AWS Foundations Benchmark, NIST SP 800-53 and SOC 2 controls. Results should be recorded so that failed checks can be investigated and evidence is available for audits.

7. Overall Security Approach

The main idea behind this architecture is to avoid depending on a single security control. Terraform provides controlled and repeatable infrastructure provisioning, while Ansible applies secure server configuration. Remote state is encrypted and locked, credentials are kept out of source code, and policy checks run before changes are applied.

The modular design also helps contain problems because different parts of the infrastructure can be managed separately. Combined with drift detection, logging, compliance checks and version-controlled rollbacks, this provides a practical way to keep cloud infrastructure secure throughout its lifecycle.

8. Architecture Diagram

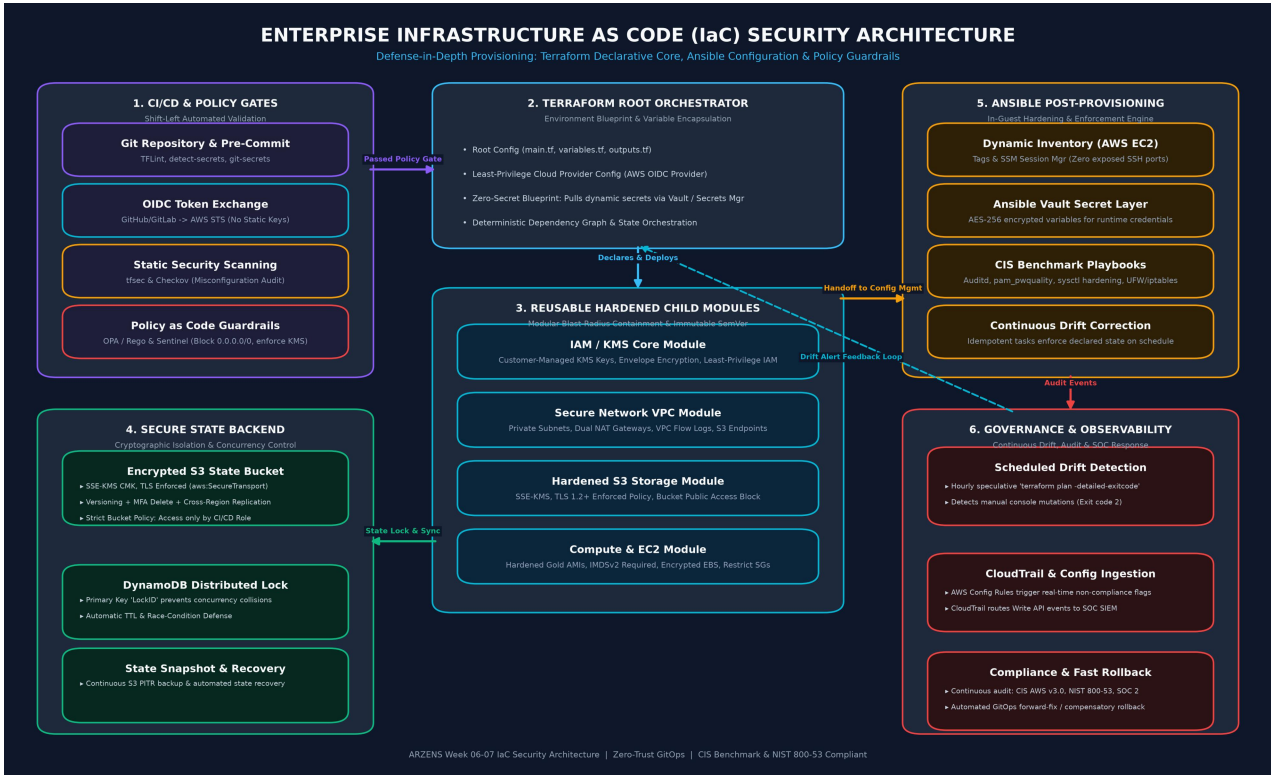


Figure 1. End-to-end IaC security architecture

9. Key Controls

Control	Implementation
Least privilege	Scoped IAM roles and short-lived OIDC credentials
State protection	S3 encryption with KMS, TLS-only access and versioning
State locking	DynamoDB LockID table
Secret management	AWS Secrets Manager / Vault and Ansible Vault
Policy enforcement	Checkov, tfsec, Trivy, OPA/Rego or Sentinel
Drift detection	Scheduled Terraform plans plus AWS Config and CloudTrail
Compliance	CIS AWS, NIST SP 800-53 and SOC 2 checks